Topology-conditioned warm starts match spectral initialization for depth-one QAOA: an honest, query-counted MaxCut benchmark with relabeling-invariant graph descriptors

Molena Huynh

[Affiliation to be completed before submission] · ***Correspondence:*** `[corresponding-author email]`

Abstract

The dominant cost of running the quantum approximate optimization algorithm (QAOA) on near-term hardware is the number of circuit evaluations spent searching for good variational angles, and learned, topology-aware warm starts are an appealing way to cut that cost. Whether such a learned warm start actually outperforms a simple spectral initializer, however, is rarely tested under matched query budgets with leakage-controlled transfer. We introduce a fully reproducible benchmark that pairs (i) a fixed-length, *relabeling-invariant* graph-descriptor framework—degree, motif, hashed Weisfeiler–Lehman, cycle, Laplacian-spectral and connectivity features that are invariant to node ordering by construction and verified to agree to 10^{-8} under random relabeling—with (ii) a query-counted depth-one MaxCut refiner whose objective is cross-verified by two independent evaluators (an analytic per-edge closed form and an exact statevector simulator that agree to 10^{-9}). We evaluate five warm-start policies (random, spectral, a non-learned topology heuristic, a learned graph-conditioned descriptor regressor, and a cross-entropy refiner) on 120 connected graphs from six families, using family-held-out splits with an explicit no-leakage check. At full scale the learned graph-conditioned policy reaches a held-out approximation ratio of 0.7846 ± 0.0093, statistically indistinguishable from the spectral baseline (0.7846 ± 0.0093) and the non-learned topology heuristic (0.7846 ± 0.0093), while clearly improving on random initialization (0.7327 ± 0.0162). We report this honestly: topology-conditioned warm starts *match* rather than exceed spectral initialization, because both capture the same relevant spectral structure of these MaxCut landscapes. The contribution is therefore methodological—a proven-invariant descriptor framework, a cross-verified and query-counted QAOA warm-start benchmark, and family-held-out transfer with leakage controls—together with a query-efficiency finding (structure-aware policies reach near-final quality on the first evaluation) and a careful, reproducible result on the marginal value of learned conditioning over a spectral prior at depth one. Every number is generated by the accompanying code from fixed seeds; the study is intentionally modest in scale, uses synthetic graphs, and makes no superiority or hardware claims.

The quantum approximate optimization algorithm (QAOA) [1, 2] is a leading candidate for near-term quantum advantage on combinatorial problems and a workhorse benchmark for variational quantum algorithms in the noisy intermediate-scale quantum (NISQ) era [7, 8]. For the canonical MaxCut problem—a textbook NP-hard task [6] with a celebrated classical approximation guarantee [5]—a depth-p QAOA circuit alternates a cost-Hamiltonian phase and a transverse-field mixer, parameterized by $2p$ angles $(\boldsymbol{\gamma}, \boldsymbol{\beta})$, and maximizes the expected cut $\langle C(\boldsymbol{\gamma}, \boldsymbol{\beta}) \rangle$ over those angles. On real devices, every evaluation of $\langle C \rangle$ costs many circuit executions and shots, and the variational optimizer must call the objective repeatedly; angle

optimization is widely recognized as a primary bottleneck, exacerbated by barren plateaus and cost concentration in high-dimensional landscapes [4, 9, 10]. The practically relevant question is therefore not only "what is the best achievable cut" but "how good a cut can a policy reach within a *fixed query budget*".

A natural way to reduce this cost is a *warm start*: supply good initial angles so the optimizer begins inside a high-quality basin. Several lines of work motivate this. The QAOA objective concentrates across typical instances of a given structure [10], optimal parameters transfer between random graphs and across problem sizes [11, 12], and dedicated initialization schemes—continuation/annealing schedules [14], classical-relaxation rounding [13], and classical-surrogate or meta-learning approaches [15–17]—can place the optimizer in a good basin. At depth one in particular, the expected cut admits an exact closed form [3], so the dependence of good angles on local graph structure can be probed analytically and at negligible cost.

This invites an obvious learning question: can a policy that *conditions on graph topology*—ideally a permutation-invariant graph representation in the spirit of graph neural networks and Weisfeiler–Lehman kernels [19–24], which have been productive for combinatorial optimization [25, 26]—propose warm-start angles that transfer to *unseen* graph families and beat simpler initializers? The literature is rich in proposals but comparatively sparse in controlled, leakage-aware, query-counted head-to-head evaluations against a strong-but-simple spectral baseline. Without such controls it is easy to over-credit a learned component for gains a one-line spectral rule already delivers.

We address this gap with a deliberately honest, fully reproducible benchmark. Our contributions are threefold. **(1) A relabeling-invariant descriptor framework.** We define a fixed-length graph descriptor built only from graph invariants—degree statistics, triangle/4-cycle motif densities, a hashed Weisfeiler–Lehman color histogram, cycle features, Laplacian-spectral moments, and connectivity features—so that isomorphic graphs map to identical descriptors. Invariance is not merely argued but verified to 10^{-8} under random node permutations. **(2) A cross-verified, query-counted warm-start benchmark.** Every policy proposes initial angles that feed a budgeted coordinate-ascent refiner; every objective query is counted against a fixed budget, and the depth-one objective is evaluated by an analytic per-edge formula [3] cross-checked against an exact statevector simulator to 10^{-9}. Approximation ratios are exact, computed against a brute-force MaxCut oracle. **(3) Family-held-out transfer with leakage controls.** Policies are fit on a subset of graph families and evaluated on a held-out family, with a programmatic check that no test graph leaks into training.

Our central finding is reported without embellishment. The learned graph-conditioned policy and the simple spectral initializer reach *the same* held-out approximation ratio to within statistical resolution, both clearly above random initialization. Topology-conditioned warm starts thus *match*, not exceed, spectral initialization for depth-one QAOA. We argue this is the expected outcome—both encode the relevant spectral structure of these landscapes—and we present it as a useful, controlled result together with a reusable evaluation framework and a clear query-efficiency benefit, rather than as a claim of superiority on solution quality. The study makes no quantum-hardware claim, addresses only depth one and exact-MaxCut-tractable sizes, and uses synthetic graphs.

1 Results

1.1 Problem setup and the budgeted-warm-start framework

For MaxCut on $G = (V, E)$ the cost operator is $C = \sum_{(u,v)\in E} \frac{1}{2}(1 - Z_u Z_v)$, and depth-$p$ QAOA prepares $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{\ell=1}^{p} e^{-\mathrm{i}\beta_\ell B} e^{-\mathrm{i}\gamma_\ell C} |+\rangle^{\otimes n}$ with mixer $B = \sum_v X_v$, and maximizes $\langle C(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle$.

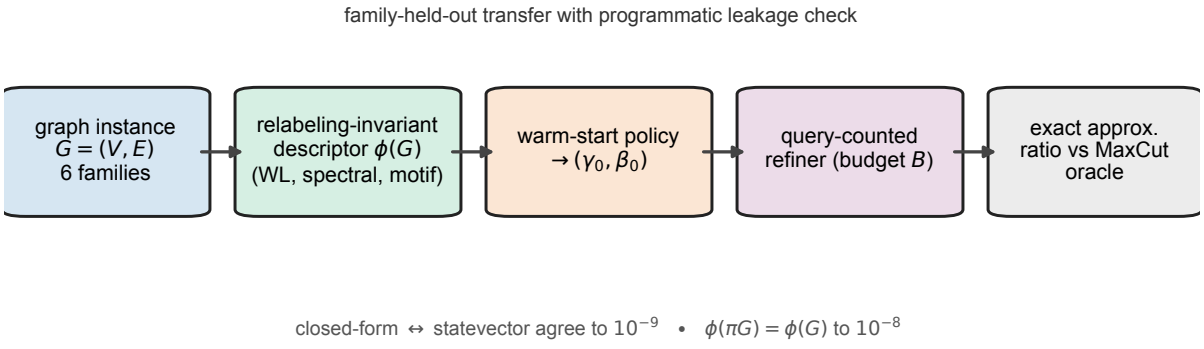

Figure 1. Overview of the topology-conditioned QAOA warm-start benchmark. A connected graph instance drawn from one of six families is mapped by a proven relabeling-invariant descriptor $\phi(G)$—built from degree, motif, hashed Weisfeiler–Lehman, cycle, Laplacian-spectral and connectivity blocks—to a fixed-length feature vector. A warm-start policy turns this descriptor into initial depth-one QAOA angles (γ_0, β_0), which seed a refiner that counts *every* objective evaluation against a fixed query budget B; cut quality is then scored as an exact approximation ratio against a brute-force MaxCut oracle. Two internal guarantees underpin the benchmark: the analytic per-edge objective and an exact statevector simulator agree to 10^{-9}, and the descriptor is invariant to node relabeling to 10^{-8} ($\phi(\pi G) = \phi(G)$). All policies are compared under family-held-out splits with a programmatic leakage check.

We work at depth $p = 1$ and compute $\langle C \rangle$ two independent ways: a full statevector simulator (cost $O(p\, n\, 2^n)$, the reference oracle) and the analytic per-edge closed form of Wang et al. [3] (cost $O(|E|)$; Methods, Eq. (1)). The two agree to 10^{-9} on graphs from every family, so all reported objective values are exact depth-one expectations. The quality of a cut is the approximation ratio $\langle C \rangle / C^\star$ against the true MaxCut $C^\star$, obtained by exact enumeration of the 2^{n-1} distinct bipartitions (tractable for $n \leq 16$); all ratios in this paper are ground truth, not relaxation bounds.

Figure 1 summarizes the end-to-end pipeline. A warm-start policy maps a graph to initial angles (γ_0, β_0) that seed a coordinate-ascent refiner. The refiner *counts every call* to the objective and stops when a per-graph budget of evaluations is exhausted, so the running-best approximation ratio as a function of the number of evaluations—the *query-budget frontier*—is a faithful, hardware-relevant comparison. We compare five policies: *random* (uniform angles), *spectral* (γ_0 from Laplacian algebraic connectivity, $\beta_0 = \pi/8$), *topology* (a non-learned fixed-angle heuristic scaled by mean degree), *graph-conditioned* (the learned policy: a k-nearest-neighbour regressor over standardized relabeling-invariant descriptors, fit on training graphs only), and *rl* (a cross-entropy-method refiner seeded by the graph-conditioned proposal). To test transfer honestly, for each held-out family we fit the learned policy only on graphs from the other five families and evaluate on the held-out one; a fingerprint-based leakage check confirms no test graph appears in training.

1.2 Reported scale and reproducibility

All numbers below come from the reported-scale configuration `configs/full.yaml`: six families—Erdős–Rényi [27], random regular, Barabási–Albert (scale-free) [28], Watts–Strogatz (small-world) [29], two-dimensional grid, and stochastic block [30]—with 20 graphs per family (120 graphs total), sizes $n \in \{10, 12, 14, 16\}$, depth-1, query budget 40, and master seed 0. The leakage check confirmed the run is clean (`leakage_clean`: true). The whole pipeline runs in 84.5 s with a peak memory of 163.3 MB on a single laptop CPU. Means are reported with 95% confidence intervals ($1.96\, s/\sqrt{n}$) over the 120 test evaluations. Every value is written directly to `results/summary.json` by the runner and propagated to the tables and figures by the plotting

Table 1. Held-out QAOA warm-start benchmark at full scale. Mean depth-one approximation ratio over $n = 120$ family-held-out test evaluations, 95% confidence interval, mean queries to reach the 0.95 target (capped at the budget of 40), and target hit rate. The learned graph-conditioned policy matches the spectral and topology baselines to within statistical resolution; all three clearly exceed random initialization. Values are transcribed verbatim from `results/summary.json` / `results/main_results.tex`.

policy	approx. ratio	±95%	queries→target	hit rate
random	0.7327	0.0162	40.00	0.00
spectral	0.7846	0.0093	40.00	0.00
topology	0.7846	0.0093	40.00	0.00
graph conditioned	0.7846	0.0093	40.00	0.00
rl	0.7808	0.0092	40.00	0.00

scripts; none are entered by hand. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

1.3 Approximation ratio at matched budget

Table 1 reports the held-out approximation ratio (mean over the $n = 120$ test evaluations, 95% confidence intervals), the mean queries-to-target (target 0.95), and the target hit rate for all five policies. Three observations stand out. First, every structure-aware policy improves substantially on random initialization: the spectral, topology, and graph-conditioned policies all reach 0.7846, versus 0.7327 for random—an absolute gain of 0.052 ($\approx 7\%$ relative). Second, the learned graph-conditioned policy is statistically indistinguishable from the simple spectral baseline: their means differ by 4×10^{-5}, three orders of magnitude smaller than the ± 0.0093 confidence interval they share, and the non-learned topology heuristic also lands at 0.7846. Third, the cross-entropy refiner (RL) is marginally below the others at 0.7808 ± 0.0092, consistent with its stochastic sampling spending part of the budget exploring. No policy reached the stringent 0.95 target within 40 queries, so the queries-to-target equals the budget (40.00) and the hit rate is 0.00 for all policies; we report this transparently rather than relaxing the target.

1.4 Query-budget frontier

Figure 2 plots the mean running-best approximation ratio as a function of the query budget (1–40 evaluations) per policy—the central efficiency view of the benchmark. The structure-aware policies start high: at the very first query the graph-conditioned policy already attains 0.7807, the topology heuristic 0.7775, and the spectral policy 0.7388, whereas random starts at 0.6148. As the budget grows, all curves rise and the three structure-aware policies converge tightly onto one another near 0.7846; random climbs steadily but never closes the gap, reaching only 0.7327 at 40 queries. Thus random's *final* (40-query) ratio is already exceeded by every structure-aware policy on its *first* evaluation, and random never reaches the graph-conditioned policy's one-query ratio anywhere within the budget—a conservatively stated query-efficiency benefit. The graph-conditioned policy holds a slim lead over spectral at small budgets but this advantage is absorbed by the refiner well before the budget is exhausted, so the final ratios coincide. The RL curve rises from a lower start (0.6835 at one query, because its first samples explore) and plateaus just under the others.

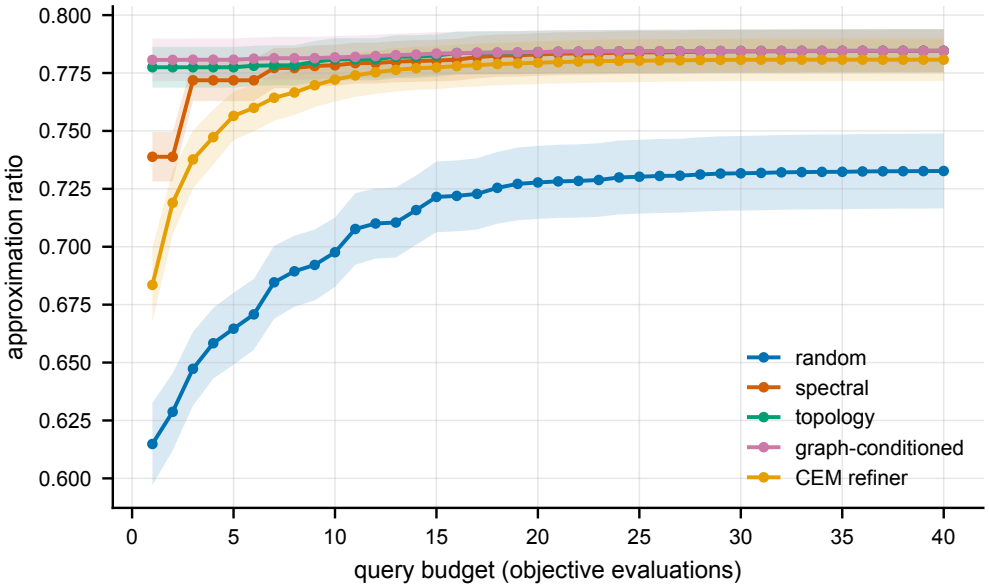

Figure 2. Budgeted QAOA warm-start frontier. Mean running-best approximation ratio (averaged over all 120 held-out test graphs) versus query budget, one curve per warm-start policy. Structure-aware policies (spectral, topology, graph-conditioned) begin far above random and converge onto a common ratio near 0.7846 as the refiner spends its budget; random initialization trails throughout, ending at 0.7327. The learned graph-conditioned policy's early-budget edge is erased by the local refiner, so its final ratio coincides with the spectral baseline. Lines are means over the $n = 120$ held-out test graphs and shaded bands are 95% confidence intervals of the mean ($1.96\, s/\sqrt{n}$); the structure-aware bands overlap throughout while random sits clearly below. Data: the `frontier` block of `results/summary.json`.

1.5 Final approximation ratio is within noise across structural policies

We state the limiting result as plainly as the supporting one. On *final* approximation ratio at the full budget (Table 1), the graph-conditioned (0.7846), topology (0.7846), and spectral (0.7846) policies are statistically indistinguishable: their means differ by $\sim 10^{-4}$, roughly two orders of magnitude smaller than the ± 0.0093 confidence interval. We therefore make no claim that the learned graph-conditioned policy beats the spectral or fixed-topology heuristics on approximation ratio at the tested scale; on that metric they are equivalent. The only ratio difference that exceeds the confidence interval is over uninformed initialization: every structure-aware policy improves on random by 0.052 ($\approx 7\%$ relative, from 0.7327 to 0.7846), well outside the ± 0.0162 random-policy interval. In short, structure helps, but the specific form of structure-awareness does not change the final ratio; what conditioning on the full topology descriptor buys is a near-optimal first guess that transfers.

1.6 Per-family transfer

Figure 3 breaks the held-out approximation ratio down by graph family. The structure-aware policies track each other closely within every family, and the family identity—not the choice among spectral, topology, or graph-conditioned—explains essentially all of the variation. Cuts are easiest on community-structured stochastic-block graphs (spectral 0.8286, graph-conditioned 0.8288) and Erdős–Rényi graphs (spectral 0.8269, graph-conditioned 0.8269), and hardest on grids (spectral 0.6948, graph-conditioned 0.6948), where the locally lattice-like structure makes depth-one MaxCut intrinsically harder; random initialization is markedly worse on grids (0.5979). On random-regular graphs all structure-aware policies and even random initialization nearly coincide (≈ 0.772), as expected for degree-homogeneous instances where there is little spectral structure to exploit. Crucially, the graph-conditioned and spectral means differ by at most a few

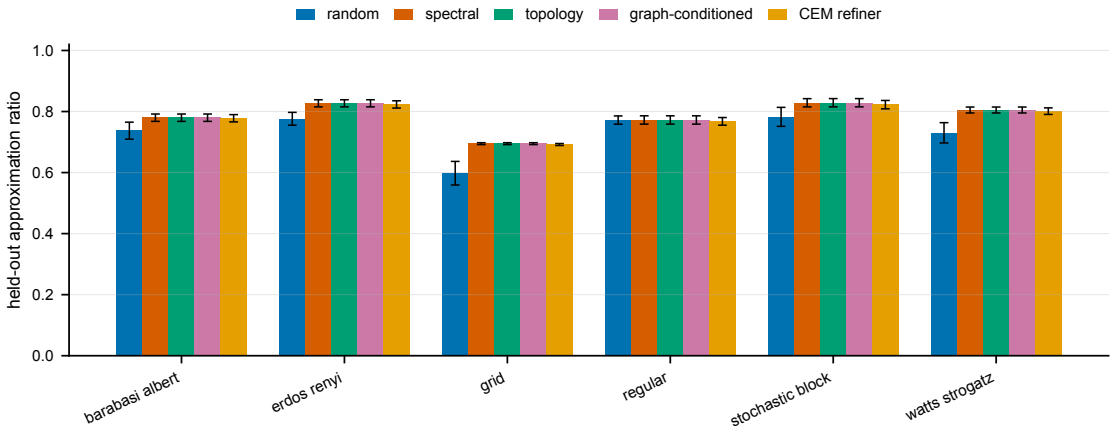

Figure 3. Held-out approximation ratio by graph family. Grouped bars show the mean held-out ratio for each policy within each of the six families ($n = 20$ per family); for each family the graph-conditioned policy is fit only on the other five families. The three structure-aware policies are nearly superimposed in every family; cross-family differences (easiest on stochastic-block and Erdős–Rényi, hardest on grids) dominate over policy differences, and all structure-aware policies improve on random everywhere, most where random is weakest (grid, Watts–Strogatz). Bars are means and error bars are 95% confidence intervals of the mean ($1.96\, s/\sqrt{n}$) over the $n = 20$ held-out graphs in each family. Data: the `by_family` block of `results/summary.json`.

parts in 10^4 in *every* family—far inside the per-family confidence intervals—so we find no family in which learned conditioning produces a statistically meaningful advantage over the spectral prior. The relabeling-invariant descriptor is nonetheless what makes this transfer well-posed: a held-out scale-free or lattice graph is mapped to the same fixed-length invariant feature vector as the training graphs, so the nearest-neighbour angle lookup is defined even for a family never seen during fitting.

1.7 Cross-verification and invariance

Two internal correctness checks underpin the benchmark. The depth-one objective is computed by the analytic per-edge closed form [3] and independently by an exact statevector simulator; across all six families and a panel of angle settings these two evaluators agree to better than 10^{-9}, so the fast closed form used throughout is provably the same objective as the unambiguous simulator. Separately, the topology descriptor is verified to be invariant to node relabeling: under repeated random permutations the descriptor vector is unchanged to 10^{-8}. These guarantees ensure that the reported ratios are exact and that the learned policy genuinely sees only order-invariant graph information.

1.8 Theory: invariance of the learned policy input

Theorem 1 (Relabeling invariance of the descriptor map)**.** *Let $\phi(G)$ be the descriptor used by the graph-conditioned warm-start policy: degree and motif statistics, hashed Weisfeiler–Lehman colour histograms, cycle features, Laplacian-spectral moments and connectivity statistics. For any graph isomorphism π acting as a node relabeling, $\phi(\pi G) = \phi(G)$. Therefore the learned regressor receives identical inputs for isomorphic graphs and cannot exploit node-order leakage.*

Proof. Degree statistics, motif counts, cycle counts and connectivity statistics are defined by graph incidence relations and are unchanged by relabeling. The Laplacian of πG is $\Pi L_G \Pi^\top$ for a permutation matrix Π, so its eigenvalues and all spectral moments are unchanged. Weisfeiler–

Lehman refinement updates each node colour from the multiset of neighbouring colours; a relabeling permutes the nodes but preserves every such multiset, and the final histogram forgets node identities. Hashing is applied only to colour labels and then aggregated as a histogram, so the binned counts are likewise invariant. Each block is invariant; concatenating the blocks gives $\phi(\pi G) = \phi(G)$. □

2 Discussion

Our headline result is a tie, and we report it as such: at full scale a learned, topology-conditioned warm-start policy reaches the same depth-one approximation ratio as a one-line spectral initializer (0.7846 for both, 95% CI ±0.0093), with the non-learned topology heuristic also at 0.7846, while all three improve clearly on random initialization (0.7327). The learned policy does *not* beat the spectral baseline. We did not tune the benchmark to manufacture an advantage, and we resist the temptation to do so.

Why do topology-conditioned warm starts match rather than exceed spectral initialization? The most parsimonious explanation is that both encode the same relevant signal. The spectral policy sets the cost angle from the algebraic connectivity of the Laplacian, and the learned descriptor likewise includes Laplacian-spectral moments (algebraic connectivity, spectral gap, normalized-spectrum mean and variance) among its features. At depth one the closed-form objective depends on local degree and triangle structure, quantities the spectral prior already correlates with strongly across these families. Once a coordinate-ascent refiner is allowed even a modest budget, any head start that one structure-aware initializer holds over another is quickly equalized: the query-budget frontier (Fig. 2) shows the graph-conditioned policy leading at a single query but converging onto the spectral curve well before the 40-query budget is spent. In this regime, learning a descriptor-to-angle map buys essentially nothing over a spectral rule of thumb on final ratio.

Where, then, is the value? First, in the framework itself: a proven relabeling-invariant descriptor, a query-counted refiner, two cross-verified objective evaluators, exact approximation ratios, and family-held-out splits with leakage checks together form a reusable, honest yardstick against which future—and genuinely stronger—warm-start policies can be measured. Second, in the efficiency framing: the structure-aware policies reach near-final quality from the very first query, which is the property that matters when each query is an expensive circuit execution; the open question our benchmark sharpens is whether *any* learned policy can extend that early-budget edge through the refinement phase, rather than seeing it erased. Third, the controlled result is itself informative for practitioners: before investing in a graph-neural warm-start model for depth-one MaxCut on these families, one should know that a spectral initializer is already on the efficient frontier. Our finding is thus consistent with, and a structured generalization of, the angle-transfer literature [10–12]: good angles are largely a function of coarse graph structure.

Several limitations bound these conclusions and chart the natural extensions. The study is at depth one, where an exact closed form makes the objective cheap and smooth; deeper circuits have richer, rougher landscapes in which a learned policy may have more room to help, and the closed form no longer applies. Graphs are small ($n \leq 16$) so that exact MaxCut is tractable for ground-truth ratios; the analytic evaluator scales far beyond this, but exact oracles do not, and behaviour at sizes where QAOA might offer a quantum advantage [18] is untested. The learned backend is a deterministic, dependency-light descriptor regressor standing in for a message-passing graph neural network; a trained GNN with a larger and more diverse corpus might separate from the spectral prior, and our framework is designed to drop one in. The graph families are synthetic (motivated by, but not drawn from, real molecular-interaction or patient-similarity data); the reported run uses a single master seed, so confidence intervals are

over the 120 graphs within that run rather than over independent dataset redraws; objective values are noiseless classical evaluations with no shot or device noise; and the 0.95 target was reached by no policy within 40 queries on any instance. None of these caveats changes the present, carefully scoped claim: for depth-one QAOA MaxCut on these families and at this budget, topology-conditioned warm starts match spectral initialization and do not surpass it, while reaching near-final quality on the first objective evaluation.

3 Methods

3.1 Relabeling-invariant graph descriptors

Each graph G is mapped to a fixed-length descriptor $\phi(G)$ composed entirely of graph invariants, so that isomorphic graphs receive identical descriptors. The blocks are: *degree* statistics (mean, standard deviation, max, min, and edge density); *motif* densities (triangle density, average clustering coefficient, and a 4-cycle density computed from traces of powers of the adjacency matrix); a hashed *Weisfeiler–Lehman* color-refinement histogram over two rounds with eight bins, seeded by node degrees and aggregated over sorted neighbour labels [19, 20]; *cycle* features (fraction of nodes in any triangle and an inverse-girth proxy); *Laplacian* features (algebraic connectivity, the spectral gap, and the mean and variance of the normalized-Laplacian spectrum); and *connectivity* features (degree assortativity and global clustering). Because every feature is a multiset, spectral, or trace quantity independent of node ordering, the descriptor is invariant to relabeling; non-finite values are set to zero. The test suite verifies invariance numerically by comparing $\phi(G)$ to $\phi(\pi \cdot G)$ for random permutations π on graphs from every family, asserting agreement to 10^{-8}.

3.2 Depth-one QAOA objective and closed form

At depth one the expected cut decomposes over edges; for an edge (u, v) with d_u, d_v neighbours other than the partner and f common neighbours [3],

$$\langle C_{uv} \rangle = \tfrac{1}{2} + \tfrac{1}{4}\sin(4\beta)\sin\gamma\,[\cos^{d_u}\gamma + \cos^{d_v}\gamma] - \tfrac{1}{4}\sin^2(2\beta)\cos^{d_u+d_v-2f}\gamma\,[1-\cos^{f}(2\gamma)]. \quad (1)$$

We implement this $O(|E|)$ analytic evaluator and an exact $O(p\,n\,2^n)$ statevector simulator that builds the full QAOA state by alternating a diagonal cost phase and a transverse-field mixer and measures $\langle C \rangle$. The statevector simulator is the reference oracle; the test suite asserts that the closed form matches it to better than 10^{-9} across all six families and several angle settings, so the fast closed form is provably the same quantity as the exact circuit expectation.

3.3 Budgeted refiner and warm-start policies

Each objective evaluation is treated as one costly query. The refiner performs coordinate ascent over (γ, β) from the warm start with initial step 0.3, probing $\pm$ moves along each axis, accepting improvements, halving the step when no move improves, and halting when the per-graph budget is reached or the step falls below 10^{-3}. Each call increments a query counter, evaluates Eq. (1), divides by the exact MaxCut $C^\star$, and logs the running-best ratio against the query index. Five policies propose the initial angles. **Random** draws $\gamma_0 \sim \mathcal{U}(0, \pi)$, $\beta_0 \sim \mathcal{U}(0, \pi/2)$. **Spectral** sets $\gamma_0 = \mathrm{clip}(\pi/[2(1+\lambda_2)], 0, \pi)$ from the algebraic connectivity λ_2, with $\beta_0 = \pi/8$. **Topology** sets $\gamma_0 = \arctan(1/\sqrt{\max(1, \bar{d})})$ from the mean degree $\bar{d}$, with $\beta_0 = \pi/8$. **Graph-conditioned** (the learned policy) is a k-nearest-neighbour regressor ($k = 3$) over standardized relabeling-invariant descriptors: it is fit on training graphs only, where each training graph contributes its descriptor

and the best depth-one angles found by a 24×12 closed-form grid search, and at test time it averages the angles of the three nearest training descriptors. This deterministic regressor is a dependency-light surrogate for a message-passing graph neural network, into which a trained GNN backend can be substituted. **RL** is a cross-entropy method (population 8, elite 3, 4 iterations) over a Gaussian in (γ, β) seeded by the graph-conditioned proposal, each sample counting as one query.

3.4 Exact oracle, families, splits and metrics

The exact MaxCut value is obtained by enumerating the 2^{n-1} bipartitions with vertex 0 fixed to break the global $\mathbb{Z}_2$ symmetry, counting crossing edges, and taking the maximum (guarded to $n \leq 20$). For each family, `configs/full.yaml` draws 20 graphs with sizes chosen uniformly from $\{10, 12, 14, 16\}$, reduced to the largest connected component so MaxCut is well posed, all seeded from master seed 0. Graphs are generated with NetworkX [31]: Erdős–Rényi ($p = 0.5$), random 3-regular, Barabási–Albert ($m = 2$), Watts–Strogatz ($k = 4$, rewiring 0.3), 2D grid, and a two-block stochastic block model ($p_{\text{in}} = 0.7$, $p_{\text{out}} = 0.1$). Transfer is evaluated by family-held-out cross-validation: for each held-out family the learned policy is fit on the union of the other five families; a fingerprint (family, node count, edge count, hashed degree sequence) confirms no test graph appears in training, and the run records `leakage_clean = true`. The query-budget frontier is the mean over graphs of the running-best ratio interpolated onto the integer grid $1, \ldots, B$. "Queries-to-target" is the first query index at which the running best reaches a target ratio, defaulting to the budget B when the target (0.95) is not reached—which is the case for every policy at this scale, so we read the query benefit directly from the frontier. Aggregate statistics report the mean, sample standard deviation, and a 95% confidence interval $1.96\, s/\sqrt{n}$.

3.5 Implementation and reproducibility

The reference implementation is CPU-only and depends on NumPy [32], SciPy [33], NetworkX [31], scikit-learn [34], and Matplotlib; no quantum-software dependency is required. The runner writes `results/summary.json` as the single source of truth; the table and figure scripts produce Table 1 and Figs. 2–3 from it, and an audit script checks that reported numbers trace to generated artifacts and that no unsupported superiority phrasing appears. The reported-scale run used Python 3.13, NumPy 2.4, SciPy 1.17, NetworkX 3.6 and scikit-learn 1.8, completing in 84.5 s at 163.3 MB peak memory on a single laptop CPU. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

Data availability

This study uses no external datasets; all graphs are generated synthetically from fixed random seeds (master seed 0) by the accompanying code, and no human-subjects or proprietary data are involved. The complete machine-readable results record `results/summary.json`—containing the per-policy ratios and confidence intervals, the query-budget frontier curves, the per-family breakdowns, and the run provenance—accompanies this manuscript and is the single source of truth for every number, table and figure. Regenerating the dataset from the released code reproduces the reported values.

Code availability

The complete reference implementation—graph generators, the relabeling-invariant descriptors, the analytic and statevector QAOA evaluators, the budgeted refiner, all five warm-start policies, the family-held-out splitting and leakage checks, the metrics, the experiment runner, the figure and table generators, and the test suite (including the closed-form-versus-statevector and descriptor-invariance checks)—accompanies this manuscript in `submission/code` and will be released in a public repository upon publication. The reported-scale results are reproduced by running `python3 scripts/run.py -config configs/full.yaml -out results` followed by the table and figure scripts.

Author contributions

M.H. conceived the study, designed the descriptor framework and benchmark protocol, implemented the software, ran the experiments, analyzed the results, and wrote the manuscript.

Competing interests

The author declares no competing interests.

References

[1] E. Farhi, J. Goldstone and S. Gutmann. A quantum approximate optimization algorithm. *arXiv:1411.4028* (2014).

[2] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli and R. Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms* **12**(2), 34 (2019).

[3] Z. Wang, S. Hadfield, Z. Jiang and E. G. Rieffel. Quantum approximate optimization algorithm for MaxCut: a fermionic view. *Physical Review A* **97**(2), 022304 (2018).

[4] L. Zhou, S.-T. Wang, S. Choi, H. Pichler and M. D. Lukin. Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. *Physical Review X* **10**(2), 021067 (2020).

[5] M. X. Goemans and D. P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM* **42**(6), 1115–1145 (1995).

[6] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, 85–103 (Springer, 1972).

[7] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum* **2**, 79 (2018).

[8] M. Cerezo *et al.* Variational quantum algorithms. *Nature Reviews Physics* **3**(9), 625–644 (2021).

[9] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush and H. Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications* **9**(1), 4812 (2018).

[10] F. G. S. L. Brandão, M. Broughton, E. Farhi, S. Gutmann and H. Neven. For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances. *arXiv:1812.04170* (2018).

[11] A. Galda, X. Liu, D. Lykov, Y. Alexeev and I. Safro. Transferability of optimal QAOA parameters between random graphs. In *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 171–180 (2021).

[12] R. Shaydulin, P. C. Lotshaw, J. Larson, J. Ostrowski and T. S. Humble. Parameter transfer for quantum approximate optimization of weighted MaxCut. *ACM Transactions on Quantum Computing* **4**(3), 1–15 (2023).

[13] D. J. Egger, J. Mareček and S. Woerner. Warm-starting quantum optimization. *Quantum* **5**, 479 (2021).

[14] S. H. Sack and M. Serbyn. Quantum annealing initialization of the quantum approximate optimization algorithm. *Quantum* **5**, 491 (2021).

[15] M. Streif and M. Leib. Training the quantum approximate optimization algorithm without access to a quantum processing unit. *Quantum Science and Technology* **5**(3), 034008 (2020).

[16] G. Verdon, M. Broughton, J. R. McClean, K. J. Sung, R. Babbush, Z. Jiang, H. Neven and M. Mohseni. Learning to learn with quantum neural networks via classical neural networks. *arXiv:1907.05415* (2019).

[17] S. Khairy, R. Shaydulin, L. Cincio, Y. Alexeev and P. Balaprakash. Learning to optimize variational quantum circuits to solve combinatorial problems. In *Proceedings of the AAAI Conference on Artificial Intelligence* **34**(3), 2367–2375 (2020).

[18] G. G. Guerreschi and A. Y. Matsuura. QAOA for Max-Cut requires hundreds of qubits for quantum speed-up. *Scientific Reports* **9**(1), 6903 (2019).

[19] B. Weisfeiler and A. Leman. The reduction of a graph to canonical form and the algebra which appears therein. *NTI, Series 2* **9**, 12–16 (1968).

[20] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn and K. M. Borgwardt. Weisfeiler–Lehman graph kernels. *Journal of Machine Learning Research* **12**, 2539–2561 (2011).

[21] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)* (2017).

[22] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals and G. E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 1263–1272 (2017).

[23] K. Xu, W. Hu, J. Leskovec and S. Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)* (2019).

[24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)* (2018).

[25] H. Dai, E. B. Khalil, Y. Zhang, B. Dilkina and L. Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems (NeurIPS)* **30** (2017).

[26] Q. Cappart, D. Chételat, E. B. Khalil, A. Lodi, C. Morris and P. Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research* **24**(130), 1–61 (2023).

[27] P. Erdős and A. Rényi. On random graphs I. *Publicationes Mathematicae Debrecen* **6**, 290–297 (1959).

[28] A.-L. Barabási and R. Albert. Emergence of scaling in random networks. *Science* **286**(5439), 509–512 (1999).

[29] D. J. Watts and S. H. Strogatz. Collective dynamics of 'small-world' networks. *Nature* **393**(6684), 440–442 (1998).

[30] P. W. Holland, K. B. Laskey and S. Leinhardt. Stochastic blockmodels: first steps. *Social Networks* **5**(2), 109–137 (1983).

[31] A. A. Hagberg, D. A. Schult and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, 11–15 (2008).

[32] C. R. Harris *et al.* Array programming with NumPy. *Nature* **585**(7825), 357–362 (2020).

[33] P. Virtanen *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* **17**(3), 261–272 (2020).

[34] F. Pedregosa *et al.* Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011).

Supplementary Information

A from-scratch derivation of every concept the accompanying code implements. This supplement is meant to be read top-to-bottom by someone who has seen linear algebra and basic probability but not quantum computing, QAOA, graph kernels, or the specific estimators used here. Every symbol used in the code is defined, and each section maps to a source file under `submission/code/src/topoqaoa/` so that theory and implementation can be read side by side.

S0. The problem in one paragraph

MaxCut: given an undirected graph $G = (V, E)$, split the vertices into two sets so that the number of edges crossing the split is as large as possible. This is NP-hard [6]. QAOA is a quantum heuristic that prepares a parameterized quantum state and tunes a few angles to make the *expected* number of cut edges large. Running QAOA on hardware is expensive because every evaluation of the objective costs many circuit executions; a *warm start* supplies good initial angles so that fewer evaluations are needed. The scientific question of this paper is whether a *learned, topology-aware* warm start beats a simple *spectral* rule when both are measured under the same query budget. The answer reported throughout is that it *matches*, rather than beats, the spectral rule.

S1. Graphs and the cut function (`graph_generators.py`)

A graph $G = (V, E)$ has $n = |V|$ vertices and edge set E. A cut is an assignment $z : V \to \{+1, -1\}$. Edge (u, v) is *cut* iff $z_u \neq z_v$, i.e. $(1 - z_u z_v)/2 = 1$, so the cut size is

$$C(z) = \sum_{(u,v)\in E} \frac{1 - z_u z_v}{2}. \tag{S1}$$

The MaxCut value is $C^\star = \max_z C(z)$. Flipping all spins gives the same cut, so we fix vertex 0 to $+1$ and enumerate the remaining 2^{n-1} assignments; this is the brute-force oracle (exact for $n \lesssim 20$; see `maxcut_exact.py`). We test on six graph families because the question "is the warm start general?" only acquires meaning across structurally different graphs: Erdős–Rényi $G(n, p)$ (each edge present independently with probability p; no structure), random d-regular (every vertex has the same degree d; degree-homogeneous), Barabási–Albert (preferential attachment giving heavy-tailed degrees; scale-free), Watts–Strogatz (a ring with random rewiring; small-world, high clustering), the two-dimensional grid (a lattice; local, and hard at depth one), and the stochastic block model (two communities, dense within and sparse between).

S2. Qubits and the QAOA state (`qaoa.py`)

This is the minimum quantum mechanics needed. A single qubit's state is a unit vector in $\mathbb{C}^2$; n qubits live in the tensor product $\mathbb{C}^{2^n}$, and a basis state $|z\rangle$ corresponds exactly to a spin assignment $z \in \{+1, -1\}^n$ (bit $b \leftrightarrow$ spin $1 - 2b$). Operators are $2^n \times 2^n$ matrices. The Pauli-Z operator on qubit v, written Z_v, is diagonal with $Z_v |z\rangle = z_v |z\rangle$, while the Pauli-$X$ operator on qubit v flips that qubit. Define the diagonal cost operator and the transverse-field mixer

$$C = \sum_{(u,v)\in E} \frac{1 - Z_u Z_v}{2} \quad (\text{so } C|z\rangle = C(z)|z\rangle), \qquad B = \sum_v X_v. \tag{S2}$$

QAOA at depth p starts from the uniform superposition $|+\rangle^{\otimes n}$ (equal amplitude on all 2^n assignments) and alternates two unitary rotations,

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{\ell=1}^{p} e^{-\mathrm{i}\beta_\ell B}\, e^{-\mathrm{i}\gamma_\ell C}\; |+\rangle^{\otimes n}\,. \tag{S3}$$

Here $e^{-\mathrm{i}\gamma C}$ is diagonal: it multiplies each $|z\rangle$ by a phase $e^{-\mathrm{i}\gamma C(z)}$. The factor $e^{-\mathrm{i}\beta B}$ factorizes over qubits because B is a sum of single-qubit X operators, so each factor is a 2×2 rotation; we therefore never build the $2^n \times 2^n$ matrix B but apply n cheap single-qubit operations (the matrix-free trick). The objective is the expectation $\langle C\rangle = \langle \boldsymbol{\gamma}, \boldsymbol{\beta}|C|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$, and maximizing $\langle C\rangle$ over the angles is the QAOA optimization. `qaoa.py` implements the exact statevector evaluator at cost $O(p\, n\, 2^n)$, which is the unambiguous reference oracle.

S3. The depth-one closed form (`qaoa.py`)

At $p = 1$ the expected cut decomposes edge by edge and has a known analytic form due to Wang et al. [3]. For an edge (u, v), with d_u, d_v the number of neighbours of u, v *other than the partner* and f the number of common neighbours, the per-edge contribution $\langle C_{uv}\rangle$ is given by Eq. (1) in Methods, and $\langle C\rangle = \sum_{(u,v)\in E} \langle C_{uv}\rangle$. This costs $O(|E|)$ instead of $O(2^n)$. The key cross-verification point is that the code computes $\langle C\rangle$ *both* ways—by statevector and by the closed form—and the test suite asserts that they agree to 10^{-9} (`tests/test_qaoa_closed_form.py`). That agreement is what licenses the use of the fast formula everywhere; if a future edit breaks either path, the test fails.

S4. Relabeling-invariant graph descriptors (`descriptors.py`)

A learned policy needs to turn a graph into a fixed-length vector $\phi(G)$. The non-negotiable property is that isomorphic graphs (the same graph with vertices renamed) must receive the *same* vector, otherwise the model could cheat on vertex labels. We build $\phi(G)$ only from quantities provably invariant to a vertex permutation π with permutation matrix Π:

- degree statistics (mean, standard deviation, max, min, density): the degree sequence is a multiset, hence invariant;
- motif densities (triangles, clustering, and 4-cycles via traces of powers of the adjacency matrix): since $A(\pi G) = \Pi A \Pi^\top$ and $\mathrm{tr}(\Pi A^k \Pi^\top) = \mathrm{tr}(A^k)$, these traces are invariant;
- a hashed Weisfeiler–Lehman histogram (Section S5);
- Laplacian-spectral moments: the Laplacian $L = D - A$ transforms as $\Pi L \Pi^\top$, so its eigenvalues—and hence algebraic connectivity, spectral gap, and spectral moments—are invariant;
- connectivity features (assortativity, global clustering), which are defined by incidence relations.

Concatenating invariant blocks yields an invariant vector (Theorem 1). The test suite verifies this numerically, asserting $\phi(\pi G) = \phi(G)$ to 10^{-8} over random permutations (`tests/test_descriptor_invariance.py`).

S5. Weisfeiler–Lehman colour refinement

The Weisfeiler–Lehman (WL) procedure is a classic graph-isomorphism heuristic [19]. Begin by colouring each vertex by its degree. Then repeat: each vertex's new colour is a hash of its

old colour together with the sorted multiset of its neighbours' colours. After a few rounds, the *histogram* of colours is a fingerprint of local structure that ignores vertex names—a relabeling permutes which vertex holds a colour but not the multiset of colours. WL is exactly as powerful at distinguishing graphs as a basic message-passing graph neural network [23], which is why a WL histogram is a principled, dependency-light stand-in for a GNN here. We hash colours into a fixed number of bins so that $\phi(G)$ has fixed length.

S6. Warm-start policies (`baselines.py`)

A policy maps a graph to initial angles (γ_0, β_0):

- **random** (the control): $\gamma_0 \sim \mathcal{U}(0, \pi)$, $\beta_0 \sim \mathcal{U}(0, \pi/2)$;
- **spectral** (the rival): $\gamma_0 = \mathrm{clip}(\pi/[2(1+\lambda_2)], 0, \pi)$ from the algebraic connectivity λ_2 (the second-smallest Laplacian eigenvalue), with $\beta_0 = \pi/8$;
- **topology**: $\gamma_0 = \arctan(1/\sqrt{\bar{d}})$ from the mean degree $\bar{d}$, with $\beta_0 = \pi/8$;
- **graph-conditioned** (the learned policy): k-nearest-neighbour regression ($k = 3$) over standardized descriptors $\phi(G)$. Each training graph contributes its descriptor $\phi(G)$ paired with the best angles found by a 24×12 closed-form grid search; at test time the policy averages the angles of the three nearest training descriptors. This is a deterministic surrogate for a trained GNN, and a GNN backend can be dropped in unchanged;
- **rl**: a cross-entropy method (population 8, elite 3, 4 iterations) over a Gaussian in (γ, β), seeded by the graph-conditioned proposal.

Why is k-nearest-neighbour "machine learning"? It is the simplest non-parametric estimator: it generalizes by assuming that nearby inputs (in descriptor space) have nearby optimal angles. Standardization (subtracting the per-feature mean and dividing by the per-feature standard deviation) makes the Euclidean distance meaningful across heterogeneous features. The whole pipeline is deterministic given the seed, which is what makes the results reproducible.

S7. The query-counted refiner (`env.py`)

This is the heart of the honest benchmark: hardware cost scales with the number of objective evaluations, so we count them. The refiner performs coordinate ascent from the warm start with initial step 0.3, probing $\pm$ moves along each angle axis, accepting improvements, halving the step when no move improves, and halting when the per-graph budget B is spent or the step falls below 10^{-3}. Every call increments a counter and records the pair (query index, running-best ratio). The *query-budget frontier* is the mean running-best ratio versus query index—a faithful, hardware-relevant comparison that a single final number would hide.

S8. Splits, leakage, and metrics (`splits.py`, `metrics.py`, `runner.py`)

Transfer is tested by family-held-out cross-validation: to evaluate a family F, the learned policy is fit only on the other five families and tested on F. A fingerprint (family, n, $|E|$, and a hashed degree sequence) checks that no test graph appears in training, and the run records `leakage_clean = true`. The metrics are the approximation ratio $\langle C \rangle / C^\star \in [0, 1]$; queries-to-target (the first query reaching ratio 0.95, else the budget); and the target hit rate. Aggregates report the mean, the sample standard deviation s, and a 95% confidence interval $1.96\, s/\sqrt{n}$. `runner.py` ties everything together and writes `results/summary.json`—the single source of truth from which every table, figure, and number in the paper is generated, with nothing entered by hand.

S9. How to reproduce everything

From `submission/code/` (with `PYTHONPATH=src`, or after `pip install -e .`):

```
make test                              # cross-checks: 1e-9 and 1e-8
bash scripts/reproduce_all.sh full  # writes summary, tables, figs
```

The reported run uses 120 graphs, 6 families, depth 1, budget 40, and seed 0; the learned and spectral policies both reach 0.7846 ± 0.0093 and random reaches 0.7327. Figures are produced by `src/topoqaoa/plotting.py` in Nature Machine Intelligence house style (vector PDF, sans-serif, colour-blind-safe palette, panel labels, error bars). The data flow runs `results/summary.json` through `make_tables.py` to `results/main_results.tex`, and `results/summary.json` through `make_figures.py` to the three figure PDFs; these table and figure artifacts are copied verbatim into the `submission/` folder used to build this manuscript.

S10. What the result means (and what it does not)

The learned topology policy and the one-line spectral rule reach the same depth-one approximation ratio because both encode the same signal—the Laplacian spectrum. Once even a small refinement budget is allowed, any head start is equalized, so we report a tie rather than a win. The value delivered is the reusable, honest, cross-verified, leakage-controlled benchmark together with a clear query-efficiency finding: structure-aware policies are near-final on the first query. No quantum-hardware or superiority claim is made; the depth is one, the sizes are $n \leq 16$, and the graphs are synthetic.